

КРАВЧЕНКО А.С.

**ПРОГРАМНАЯ ДОКУМЕНТАЦИЯ НА
“ОБЛАЧНОЕ ХРАНИЛИЩЕ КАРТИНОК”**

**РАЗРАБОТАННОЕ НА ОСНОВЕ ЯЗЫКА PYTHON
И ФРЕЙМВОРКОВ DRF и PyQt5**

**ЮЖНО-САХАЛИНСК
2026**

СОДЕРЖАНИЕ

1.ВВЕДЕНИЕ	3
2.ТЕХНИЧЕСКОЕ ЗАДАНИЕ	4
3. ОБЩАЯ ЛОГИЧЕСКАЯ СХЕМА ПРОГРАММЫ	6
4.ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ	7
5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ	7
6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНИХ ВОЗДЕЙСТВИЙ	7
7. КАК РАБОТАЕТ ПРОГРАММА	8
8. ЛОГИЧЕСКИЕ ВЗАИМОСВЯЗИ	9
9. ЛИСТИНГ ПРОГРАММЫ	10

1.ВВЕДЕНИЕ

Документация описывает разработанное программное обеспечение для передачи и получения изображений по сети с использованием клиент-серверной архитектуры. Проект реализован на языке Python с применением фреймворков PyQt5 и Django REST Framework.

Программное обеспечение состоит из двух основных частей:

1. Серверная часть на базе Django REST Framework:

- обработка HTTP-запросов;
- хранение изображений;
- предоставление REST API;
- сериализация данных.

2. Клиентская часть на базе PyQt5:

- графический интерфейс пользователя;
- выбор изображения;
- отправка изображений на сервер;
- получение списка изображений;
- отображение изображений.

Программа реализует взаимодействие между клиентом и сервером через HTTP-запросы библиотеки requests.

2.ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1. ВВЕДЕНИЕ

Наименование разработки: «Облачное хранилище изображений».

Программа предназначена для демонстрации клиент-серверного взаимодействия между приложением PyQt5 и сервером Django REST Framework.

2.2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

Программа предназначена для:

- загрузки изображений на сервер;
- хранения изображений;
- получения списка загруженных изображений;
- просмотра изображений через графический интерфейс.

2.3. ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

2.3.1. Требования к функциональным характеристикам

2.3.1.1. Требования к инструментальной среде

1. Python версии не ниже 3.10.
2. Используемые библиотеки:
 - a. PyQt5;
 - b. Django;
 - c. Django REST Framework;
 - d. Pillow;
 - e. requests.

2.3.1.2. Требования к пользовательскому интерфейсу

1. Интерфейс должен содержать:
 - a. кнопки отправки и получения изображений;
 - b. поле ввода пароля;
 - c. список изображений;
 - d. область предпросмотра изображения.
2. Пользователь должен иметь возможность:
 - a. выбрать изображение;
 - b. отправить изображение;
 - c. получить изображения с сервера;
 - d. просмотреть выбранное изображение.

2.3.1.3. Требования к реализуемым функциям

1. При запуске клиентское приложение:

- a. создает графический интерфейс;
 - b. подключает обработчики событий.
2. При выборе изображения:
 - a. открывается QFileDialog;
 - b. путь сохраняется в переменную image_path.
3. При отправке изображения:
 - a. выполняется POST-запрос к серверу;
 - b. изображение передается через multipart/form-data.
4. При получении изображений:
 - a. выполняется GET-запрос;
 - b. список изображений отображается в QListWidget.
5. При выборе изображения из списка:
 - a. изображение загружается;
 - b. выполняется отображение в QLabel.

2.3.2. Требования к надежности

1. Программа должна обрабатывать ошибки:
 - a. отсутствия изображения;
 - b. отсутствия соединения с сервером;
 - c. пустого списка изображений.
2. Интерфейс не должен завершаться аварийно при ошибках запросов.

2.3.3. Требования к составу технических средств

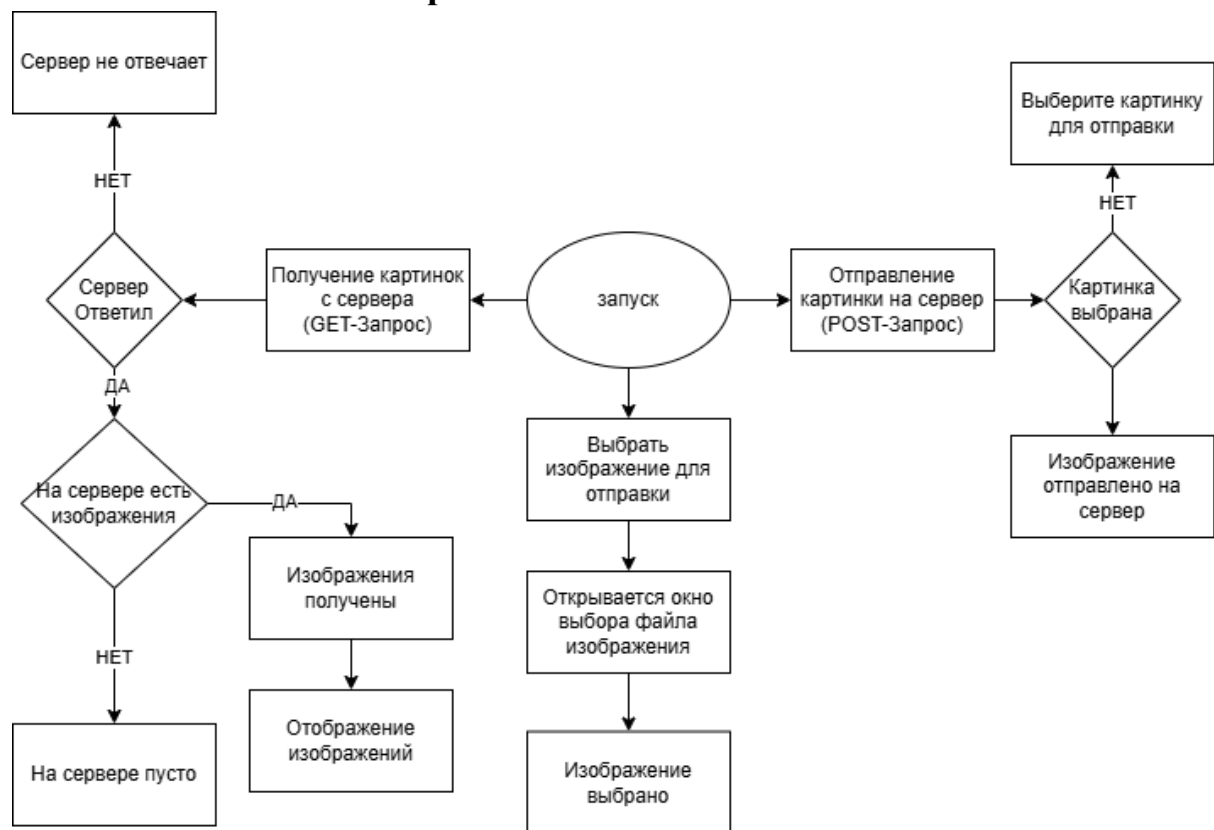
1. ОС Windows/Linux.
2. Наличие Python и необходимых библиотек.
3. Наличие сетевого соединения между клиентом и сервером.

2.3.4. Требования к совместимости

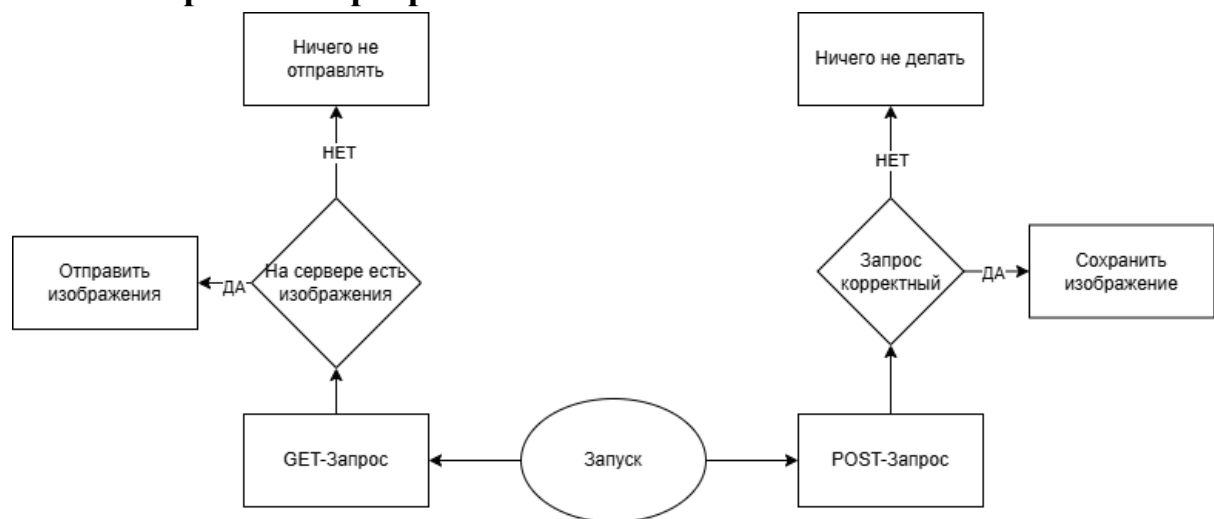
1. Сервер должен поддерживать HTTP-запросы.
2. Передача данных выполняется через REST API.

3. ОБЩАЯ ЛОГИЧЕСКАЯ СХЕМА ПРОГРАММЫ

3.1 Схема клиентского приложения



3.2 Схема работы сервера



4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

№	Функция/Класс	Описание
1	MyWindow(QWidget)	Главное окно программы
2	initUI()	Создание интерфейса

3	change_text1()	Отправка изображения
4	change_text2()	Получение изображений
5	item_clicked()	Выбор изображения из полученных
6	pick_file()	Выбор файла изображения
7	Image(models.Model)	Модель изображения
8	ImageSerializer	Сериализация данных
9	requests.post()	Отправка POST-Запроса
10	requests.get()	Отправка GET-Запроса

5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

№	Переменная	Описание
1	self.password_field.text()	Введенный пароль
2	self.image_path	Путь к изображению
3	item.text()	URL изображения
4	r.content	Содержимое ответа сервера
5	files={"image": f}	Отправляемое изображение

6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

№	Воздействие	Описание
1	self.widget3.setText()	Вывод статуса запроса
2	self.image.setPixmap()	Отображение изображений
3	QListWidgetItem()	Добавление изображения в список
4	requests.post()	Отправка изображения
5	requests.get()	Получение изображений

7. КАК РАБОТАЕТ ПРОГРАММА

7.1. Запуск программы

При запуске выполняются:

1. Запуск сервера Django: `os.system('python project/run.py')`

2. Создание QApplication.
3. Создание окна MyWindow.
4. Отображение интерфейса.

7.2. Выбор изображения

Пользователь нажимает кнопку:

```
self.image_button.clicked.connect(self.pick_file)
```

После чего:

- открывается QFileDialog;
- выбирается изображение;
- путь сохраняется в self.image_path.

7.3. Отправка изображения

Метод:

```
change_text1()
```

выполняет:

1. Проверку пароля.
2. Открытие файла изображения.
3. POST-запрос:
`requests.post("http://127.0.0.1:8000/main/images/", files={"image": f})`
4. Сервер сохраняет изображение.

7.4. Получение изображений

Метод:

```
change_text2()
```

выполняет:

1. GET-запрос:
`requests.get("http://127.0.0.1:8000/main/images/")`
2. Получение JSON-списка изображений.
3. Загрузка изображений.
4. Добавление элементов в QListWidget.

7.5. Просмотр изображения

При выборе изображения:

```
self.images_group.itemClicked.connect(self.item_clicked)
```

выполняется:

- загрузка изображения по URL;
- отображение в QLabel.

8. ЛОГИЧЕСКИЕ ВЗАИМОСВЯЗИ

8.1. Архитектура программы

Программа построена по клиент-серверной архитектуре:

Клиент:

- PyQt5 GUI;
- requests;
- обработка изображений.

Сервер:

- Django;
- Django REST Framework;
- ImageField;
- сериализация JSON.

8.2. Взаимодействие компонентов

Отправка изображения

PyQt5 → requests.post() → Django REST API → Image модель

Получение изображения

PyQt5 → requests.get() → Django REST API → JSON → QListWidget

8.3. Работа интерфейса

GUI работает через механизм сигналов и слотов PyQt5:

```
clicked.connect(...)
```

```
itemClicked.connect(...)
```

9. ЛИСТИНГ ПРОГРАММЫ

9.1. Модель Django

```
class Image(models.Model):  
    image = models.ImageField(upload_to='images/')
```

9.2. Сериализатор

```
class ImageSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Image  
        fields = ['image']
```

9.3. Отправка изображения

```
r = requests.post(  
    "http://127.0.0.1:8000/main/images/",  
    files={"image": f}  
)
```

9.4. Получение изображений

```
r = requests.get(  
    "http://127.0.0.1:8000/main/images/"  
)
```

9.5. Отображение изображения

```
self.pixmap.loadFromData(x.content)  
scaled_pixmap = self.pixmap.scaled(  
    200,  
    200,  
    QtCore.Qt.KeepAspectRatio  
)  
self.image.setPixmap(scaled_pixmap)
```